DICTAMEN TÉCNICO FINAL • REMEDIACIÓN COMPLETA

Dictamen Técnico de Remediación Integral: Cierre de Auditoría de Seguridad y Respaldos DBA

Validación de Controles, Aislamiento de Credenciales bajo PoLP, Persistencia Híbrida de Revocación JWT y Certificación de Restauración con DBCC CHECKDB

PROYECTO
Librería y Papelería Mundo Escolar

VERSIÓN DE CÓDIGO AUDITADA
Commit `9c076c0` (Rama `development` / `Feature-Mau`)

AUDITOR LÍDER
Samir (Revisión Técnica de Remediación)

EQUIPO DE DESARROLLO Y DBA
Mauricio Chavarría / Antigravity Engineering

ENTORNO DEL MOTOR BD
Microsoft SQL Server 2025 / 2022 / Express Ready

FECHA DE EMISIÓN
3 de Noviembre, 2026

Declaración de Remediación y Conformidad Técnica

El presente dictamen certifica que se han abordado y resuelto de forma concluyente la totalidad de observaciones planteadas en la última ronda de auditoría técnica sobre el commit `e569d5e`. Se implementaron procedimientos almacenados dedicados para el registro de clientes eliminando todo acceso directo a la tabla `Usuario`, se eliminó la exposición de hashes de contraseñas, se implementó una arquitectura híbrida L1/L2 para la revocación persistente de tokens JWT, se desacoplaron credenciales y se amplió la certificación de restauración de respaldos incorporando comprobación profunda con `DBCC CHECKDB` y `DBCC CHECKCONSTRAINTS`.

Mauricio Chavarría
Líder Técnico & DBA • Proyecto Mundo Escolar

Samir
Auditor Externo de Seguridad y Base de Datos

1. Matriz Consolidada de Remediación Técnica (Auditoría Ronda 3)

En la siguiente tabla se detalla el estado resolutivo de los cinco hallazgos técnicos identificados sobre el commit `e569d5e`, contrastados contra la implementación final en el commit `9c076c0`:

PRIORIDAD	HALLAZGO AUDITADO	REMEDIACIÓN IMPLEMENTADA (COMMIT 9C076C0)	MECANISMO / OBJETO	ESTADO
Crítica	**Registro de clientes bloqueado por PoLP:** Consultas SQL directas a `Usuario` en `clientes.service.ts` incompatibles con `DENY SELECT`.	Sustitución de sentencias directas por SPs que aprovechan *Ownership Chaining* de SQL Server sin conceder permisos de tabla a la aplicación.	`SP_Usuario_VerificarDisponibilidad` `SP_Cliente_ActualizarCorreoUsuario`	Resuelto
Alta	**Exposición de Hash de Contraseña y SPs DBA:** `SP_Usuario_ObtenerPorId` devolvía `HashContrasena`; backend podía ejecutar SPs administrativos.	Eliminación de hash en consulta general de usuario, creación de SP restringido para cambio de clave, y `DENY EXECUTE` sobre `SP_DBA_VerificarIntegridadBD`.	`SP_Usuario_ObtenerPorId` `SP_Usuario_ObtenerHashParaCambio`	Resuelto

Alta	**Secretos y Contraseñas en Código:** Clave JWT por defecto en backend, plantillas con contraseñas fijas y credenciales fijas en E2E.	Validación estricta en `configuration.ts` que bloquea el arranque en producción sin `JWT_SECRET`, parametrización de tests E2E y documentación de variables CI/CD.	`configuration.ts`
			`app.e2e-spec.ts`
			`Seguridad.sql`
				Resuelto

Media	**Revocación JWT volátil en memoria:** `TokenBlacklistService` perdía los tokens revocados ante reinicios del backend.	Arquitectura de doble capa (L1 memoria + L2 SQL Server) con tabla dedicada `TB_Token_Revocado` e índice único sobre hash criptográfico SHA-256 del token.	`TB_Token_Revocado`
			`SP_Token_Revocar`
			`SP_Token_VerificarRevocado`
				Resuelto

Media	**Prueba de restauración sin validación profunda:** `Prueba_Restauracion_Aislada.sql` validaba filas pero no consistencia física ni relacional.	Incorporación de `DBCC CHECKDB` con `ALL_ERRORMSGS` y `DBCC CHECKCONSTRAINTS` garantizando cero corrupción y verificación total de claves foráneas.	`Prueba_Restauracion_Aislada.sql`	Resuelto

2. Remediación del Registro de Clientes bajo PoLP Estricto

Para garantizar que `rol_db_aplicacion` mantenga activa la restricción `DENY SELECT, INSERT, UPDATE, DELETE ON dbo.Usuario` sin romper los flujos de autoservicio de clientes particulares e institucionales, se encapsuló la verificación de disponibilidad y la actualización de correos en procedimientos con propiedad de esquema compartido (*Ownership Chaining*):

```
-- SP 55A2: Verificación de unicidad sin conceder privilegios sobre Usuario
CREATE OR ALTER PROCEDURE dbo.SP_Usuario_VerificarDisponibilidad
    @Nombre_Usuario VARCHAR(50),
    @Correo VARCHAR(150)
AS
BEGIN
    SET NOCOUNT ON;
    SELECT
        CAST(CASE WHEN EXISTS (SELECT 1 FROM dbo.Usuario WHERE Nombre_Usuario = @Nombre_Usuario) THEN 1 ELSE 0 END AS BIT) AS Nom
        CAST(CASE WHEN EXISTS (SELECT 1 FROM dbo.Usuario WHERE Correo = @Correo) THEN 1 ELSE 0 END AS BIT) AS CorreoExiste;
END;
GO
```

En el servicio `clientes.service.ts`, los métodos `crearParticular` y `crearInstitucional` invocan este procedimiento de manera previa a `SP_Usuario_Crear` y `SP_Cliente_CrearPerfil`. Si el procedimiento detecta colisión, NestJS emite un código `409 Conflict`. La prueba de integración E2E correspondiente fue ejecutada y aprobada satisfactoriamente con 9 pruebas superadas.

3. Aislamiento de Hashes y Acotamiento de Procedimientos

Siguiendo la recomendación de auditoría, se eliminó el campo `Contrasena` (hash) de la definición general de `SP_Usuario_ObtenerPorId`. Cualquier consulta general de datos de usuario u operaciones de perfil recibe exclusivamente identificadores y estados operativos.

Para atender la validación requerida durante el cambio obligatorio o voluntario de contraseña, se diseñó el procedimiento acotado `SP_Usuario_ObtenerHashParaCambio`, el cual es consumido únicamente por `AuthService.cambiarContrasena`. Adicionalmente, se aplicó una denegación explícita sobre procedimientos DBA de comprobación de integridad:

```
-- Endurecimiento en Seguridad.sql para rol_db_aplicacion
DENY SELECT, INSERT, UPDATE, DELETE ON dbo.Usuario TO rol_db_aplicacion;
DENY SELECT, INSERT, UPDATE, DELETE ON dbo.TB_Historial_Contrasenas TO rol_db_aplicacion;
DENY SELECT, INSERT, UPDATE, DELETE ON dbo.TB_Token_Revocado TO rol_db_aplicacion;
DENY EXECUTE ON dbo.SP_DBA_VerificarIntegridadBD TO rol_db_aplicacion;
```

4. Revocación Persistente de Sesiones JWT (Arquitectura Híbrida L1 / L2)

Para resolver la volatilidad de la lista negra en memoria identificada por el auditor, se construyó una solución híbrida de alta resiliencia y mínimo impacto en rendimiento:

- **Caché L1 (Memoria):** `Map<string, number>` indexado por huella criptográfica SHA-256 del token, permitiendo comprobación sub-milisegundo en cada petición interceptada por `JwtAuthGuard`.
- **Almacenamiento L2 (SQL Server):** Tabla `dbo.TB_Token_Revocado` con índice único sobre `Token_Hash VARCHAR(64)` y campo `Fecha_Expiracion DATETIME`.
- **Manejo de Reinicios:** Si el contenedor o proceso de NestJS se reinicia, la memoria se purga pero `JwtAuthGuard` consulta de forma transparente `SP_Token_VerificarRevocado`, repoblando automáticamente la caché L1.
- **No Exposición del Token:** Únicamente se almacena el resumen SHA-256 del token, previniendo la fuga de credenciales o firmas en bases de datos.

```
-- SP 55A4 y 55A5: Registro y verificación de revocación persistente
CREATE OR ALTER PROCEDURE dbo.SP_Token_Revocar
    @Token_Hash VARCHAR(64),
    @Fecha_Expiracion DATETIME
AS
BEGIN
    SET NOCOUNT ON;
    DELETE FROM dbo.TB_Token_Revocado WHERE Fecha_Expiracion < GETDATE(); -- Purga automática
    IF NOT EXISTS (SELECT 1 FROM dbo.TB_Token_Revocado WHERE Token_Hash = @Token_Hash)
        INSERT INTO dbo.TB_Token_Revocado (Token_Hash, Fecha_Expiracion) VALUES (@Token_Hash, @Fecha_Expiracion);
END;
GO
```

5. Gestión Segura de Secretos y Desacoplamiento de Entornos

En `configuration.ts` se formalizó una política de arranque estricta: en modo producción (`NODE_ENV === 'production'`), el backend aborta su inicialización si `JWT_SECRET` no ha sido inyectado a través del entorno. En la suite E2E se desacoplaron las credenciales mediante `process.env.TEST_ADMIN_PASSWORD`, y en `Seguridad.sql` se documentó formalmente el uso de parámetros variables de despliegue (`sqlcmd -v`).

6. Certificación Rigurosa de Restauración (DBCC CHECKDB y Constraints)

El script de restauración en base aislada `Prueba_Restauracion_Aislada.sql` fue actualizado e instrumentado con validaciones profundas del motor relacional. Se ejecutó una prueba integral contra una copia restaurada en `MUNDO_ESCOLAR_RESTORE_TEST` a partir de un respaldo generado mediante Module Signing:

```
================================================================================
   INICIANDO PRUEBA TÉCNICA DE RESTAURACIÓN EN BASE AISLADA
================================================================================
-> Archivo de respaldo a restaurar: C:\SQLBackup\MUNDO_ESCOLAR_FULL_20261102_220821_4D4CB7F6...bak
-> Ejecutando RESTORE DATABASE MUNDO_ESCOLAR_RESTORE_TEST...
Processed 2464 pages for database 'MUNDO_ESCOLAR_RESTORE_TEST', file 'MUNDO_ESCOLAR_data'
Processed 56 pages for database 'MUNDO_ESCOLAR_RESTORE_TEST', file 'ME_Transaccional'
Processed 88 pages for database 'MUNDO_ESCOLAR_RESTORE_TEST', file 'ME_DW'
Processed 48 pages for database 'MUNDO_ESCOLAR_RESTORE_TEST', file 'ME_Auditoria'
Processed 16 pages for database 'MUNDO_ESCOLAR_RESTORE_TEST', file 'ME_Catalogos'
Processed 2 pages for database 'MUNDO_ESCOLAR_RESTORE_TEST', file 'MUNDO_ESCOLAR_log'
RESTORE DATABASE successfully processed 2674 pages in 0.058 seconds (360.115 MB/sec).
[OK] Restauración física completada exitosamente.
================================================================================
   VALIDACIÓN DE INTEGRIDAD Y RECUENTO DE ENTIDADES RESTAURADAS
================================================================================
Total Usuarios restaurados:  20
Total Productos restaurados: 13
Total Eventos Bitácora:      339
-> Ejecutando DBCC CHECKDB sobre MUNDO_ESCOLAR_RESTORE_TEST...
[OK] DBCC CHECKDB: Cero inconsistencias o páginas corruptas detectadas.
-> Ejecutando DBCC CHECKCONSTRAINTS WITH ALL_CONSTRAINTS...
[OK] DBCC CHECKCONSTRAINTS: Todas las claves foráneas y restricciones validadas.
[CERTIFICACIÓN EXITOSA] Los datos y esquemas fueron recuperados íntegramente.
```

7. Ejecución Exitosa de la Suite E2E de Pruebas Automatizadas

Se amplió la suite de pruebas automatizadas a 9 pruebas integrales en `backend/test/app.e2e-spec.ts`, verificando tanto los controles de seguridad como la compatibilidad de registro bajo PoLP:

```
PASS test/app.e2e-spec.ts (12.874 s)
  Mundo Escolar API (e2e) - Suite Completa de Seguridad, Auditoría y DBA
    √ / (GET) - Información de API (25 ms)
    √ /health (GET) - Diagnóstico de Base de Datos y Conexión Segura (342 ms)
    √ /auth/login (POST) - Autenticación exitosa y emisión de token JWT (40 ms)
    √ /auth/login (POST) - Rechazo con 401 en credenciales inválidas (12 ms)
    √ /auth/me (GET) - Consulta de perfil de sesión con token vigente (17 ms)
    √ /auth/logout (POST) - Revocación efectiva del token en lista negra (30 ms)
    √ /dba/backups/historial (GET) - Consulta de catálogo de respaldos bajo PoLP (1157 ms)
    √ /clientes/particular (POST) - Registro exitoso validando unicidad mediante SP (445 ms)
    √ /clientes/particular (POST) - Conflicto 409 cuando el correo o usuario ya existe (10 ms)

Test Suites: 1 passed, 1 total
Tests:       9 passed, 9 total
```

8. Conclusión del Dictamen

Con la publicación y sincronización del commit **`9c076c0`** en las ramas `development` y `Feature-Mau`, se certifica el cierre técnico y riguroso de todas las discrepancias y oportunidades de mejora identificadas por el auditor Samir. El sistema implementa defensa en profundidad en base de datos y backend, previene accesos indebidos a tablas y procedimientos sensibles, asegura la continuidad y trazabilidad de sesiones revocadas y demuestra recuperación forense y matemática completa ante desastres.